

Section 7.2: Asynchronous Messaging & Event-Driven Topologies

Study Set: 3 files total | **Section:** 7.*

Focus: Queue vs pub/sub vs log, event sourcing, CQRS, CDC, and exactly-once semantics for system design interviews.

Executive Overview

Asynchronous messaging decouples producers from consumers, absorbs traffic spikes, and enables resilient retry semantics. This section covers the three primary messaging topologies, event-driven data integration patterns, and the canonical exactly-once processing problem — a frequent interview deep-dive.

1. Async Messaging Topologies

1.1 Point-to-Point Message Queue

```
Producer → [Queue] → Consumer A
                ↘
                ↙ Consumer B ← competing consumers (load-balanced)
```

Delivery semantics: - Each message is consumed by **exactly one** consumer - Queue depth is the natural backpressure signal - Messages are acknowledged after processing; unacknowledged messages re-queue

RabbitMQ example:

```
# Producer — durable message (survives broker restart)
channel.basic_publish(
    exchange='',
    routing_key='task_queue',
    body=json.dumps(task_data),
    properties=pika.BasicProperties(delivery_mode=2)
)
```

```
# Consumer — manual ack prevents data loss
def process_task(ch, method, properties, body):
    task = json.loads(body)
    result = do_work(task)
    ch.basic_ack(delivery_tag=method.delivery_tag)
```

Best for: Work queues, job distribution, task retries, dead-letter patterns.

1.2 Publish/Subscribe (Fan-out)

```
Publisher → [Topic] → Subscriber A (analytics)
                  → Subscriber B (notifications)
                  → Subscriber C (audit log)
```

Delivery semantics: - Each subscriber gets its **own copy** of every message - Subscribers are independent — one slow subscriber doesn't affect others - Fan-out on write; storage cost multiplies with subscriber count

Kafka pub/sub example:

```
# Producer
producer.send('user_events', {'user_id': 123, 'event': 'order_placed'})

# Subscriber (independent consumer group)
consumer = KafkaConsumer(
    'user_events',
    group_id='analytics-processor',
    auto_offset_reset='earliest'
)
```

Best for: Notification systems, audit/analytics fan-out, event broadcasting.

1.3 Distributed Event Log (Kafka)

Kafka is a log, not a queue. Messages are **retained** after consumption — consumers track their own offset.

```
Partition 0: [msg0][msg1][msg2]...[msgN]    ← immutable, append-only
Partition 1: [msg0][msg1][msg2]...[msgN]
Partition 2: [msg0][msg1][msg2]...[msgN]
```

```
Consumer group A offset: {p0: 1000, p1: 950, p2: 1023}
Consumer group B offset: {p0: 200, p1: 180, p2: 210} ← independent
```

Key properties: - **Ordering guaranteed per partition** — across partitions, no global order -

Replay: New consumers can replay from offset 0 — enables re-processing and backfills -

Retention: Data lives for a configured TTL (hours to infinite with compaction) - **Throughput:** Millions of messages/second via batching + zero-copy

Partition rebalancing (important interview topic):

```
Consumer joins consumer group:
1. Group coordinator detects membership change
2. All consumers stop consuming ("stop the world")
3. Partitions reassigned to minimize movement
4. Consumers resume from last committed offset
```

Gap risk: messages processed between last commit and rebalance may be reprocessed → requires idempotent consumers.

1.4 Topology Comparison Matrix

Topology	Fan-out	Ordering	Replay	Scalability	Best Use Case
Queue (RabbitMQ/ SQS)	1 consumer/ msg	Per-queue	No	Good	Work distribution
Pub/Sub (SNS/Redis)	N consumers/ msg	Per-topic	Limited	Good	Broadcast events
Log (Kafka/ Kinesis)	N consumer groups	Per-partition	Yes	Excellent	Event streaming, audit

2. Event-Driven Data Integration Patterns

2.1 Event Sourcing

Core idea: Instead of storing current state, store every state *change* as an immutable event. Current state = replay of all events.

Traditional approach:

```
UPDATE accounts SET balance = 100 WHERE id = 1; ← history lost
```

Event Sourcing:

```
BalanceDebited (id=1, amount=10, ts=T1)
```

```
BalanceDebited (id=1, amount=5, ts=T2)
```

```
BalanceCredited(id=1, amount=20, ts=T3)
```

Reconstruct state: $\text{balance} = 0 - 10 - 5 + 20 = 5$

Benefits: - Complete, immutable audit trail — critical for finance, healthcare, compliance - Time-travel queries: “What was the account balance at T2?” - Natural event bus — downstream systems subscribe to state changes - Debugging: replay events to reproduce any past state

Drawbacks: - Current-state queries are expensive without pre-built projections (snapshots) - Event schema versioning is hard — old events must remain deserializable - Storage grows unboundedly without archival/snapshotting strategy

Snapshot pattern (essential for long-lived aggregates):

Every N events → persist a snapshot of current state

Replay = load latest snapshot + events since snapshot

2.2 CQRS (Command Query Responsibility Segregation)

Core idea: Separate the write model (commands) from the read model (queries). They can use different storage engines optimized for each.

Write side (Commands):

- └ Normalized schema (consistency, transactions)
- └ Event sourcing optional
 - └ Accepts: CreateOrder, UpdateOrderStatus

Read side (Queries):

- └ Denormalized projections (pre-joined, pre-aggregated)
- └ Different storage: Redis, Elasticsearch, NoSQL
 - └ Returns: OrderSummaryView, UserDashboardView

When CQRS is justified: - Read:write ratio is very skewed (e.g., 1000:1 reads) — optimize each path independently - Complex domain requiring event sourcing (CQRS pairs naturally with ES) - Separate scaling requirements (read replicas are not sufficient)

When CQRS is overkill: - Simple CRUD applications — adds significant complexity - Small teams — two models to maintain doubles cognitive load

2.3 Change Data Capture (CDC)

Core idea: Tap the database's write-ahead log (WAL) to emit every INSERT/UPDATE/DELETE as an event — without changing application code.

Debezium architecture:

```
PostgreSQL WAL → Debezium Connector → Kafka → Consumers
                ↓
        Change event: {op: "u", before: {...}, after: {...}}
```

Use cases: - Cache invalidation (invalidate Redis on DB update) - Microservice data sync without distributed transactions - Real-time search index updates (Elasticsearch sync) - Audit logging without application-layer instrumentation

CDC vs. dual-write comparison:

Approach	Atomicity	App changes	Latency	Risk
Dual-write (app writes to 2 systems)	None — can diverge	Requires code change	Low	High (partial failure)
CDC (tap WAL)	Transactional — WAL is atomic	None	Near-real-time	Low

Always prefer CDC over dual-write when the goal is keeping a secondary store in sync with a primary database.

3. Exactly-Once Processing — Deep Dive

Exactly-once is the hardest guarantee in distributed systems. Interviewers frequently probe here because it requires idempotency at every layer.

Layers of deduplication:

Layer 1 — Kafka producer idempotence:

```
producer = KafkaProducer(
    enable_idempotence=True,      # Sequence numbers prevent producer-side
    duplicates                    duplicates
```

```
        transactional_id='order-processor-1'  
    )
```

Covers: duplicate sends due to network retries from the *same* producer instance.

Layer 2 — Kafka transactions (read-process-write atomically):

```
producer.init_transactions()  
producer.begin_transaction()  
try:  
    result = process_order(event)  
    producer.send('processed_orders', result)  
    producer.send_offsets_to_transaction(offsets, consumer_group)  
    producer.commit_transaction()  
except Exception:  
    producer.abort_transaction() # Rolled back — no partial state
```

Covers: consumer-producer loop within Kafka. Does NOT cover external side effects (DB writes, emails).

Layer 3 — Idempotent consumer (essential for external side effects):

```
def process_order_event(event):  
    order_id = event['order_id']  
  
    # Fast path: Redis cache check  
    if redis.exists(f"processed_order:{order_id}"):   
        metrics.increment("duplicate_events_skipped")  
        return "Already processed"  
  
    # Slow path: DB transaction with unique constraint  
    with database.transaction():  
        existing = db.query("SELECT id FROM orders WHERE id = ?", order_id)  
        if existing:  
            return existing  
  
        order = create_order(event['data'])  
        db.insert("INSERT INTO orders ...", order)  
  
        # Mark in Redis after DB commit  
        redis.setex(f"processed_order:{order_id}", 1, ttl=86400)  
  
    producer.send('inventory_reserved', order_id)  
    return order
```

Layer 4 — Database idempotency key:

```
CREATE TABLE orders (  
    id                UUID PRIMARY KEY,
```

```

    idempotency_key TEXT UNIQUE,    -- External event ID as unique constraint
    status          TEXT,
    created_at      TIMESTAMPTZ
);
-- Duplicate INSERT fails with unique constraint violation → safe to catch and ignore

```

Recovery scenarios:

Failure	Result without idempotency	Result with idempotency
Consumer crashes mid-processing	Reprocessed from last offset → double charge	Idempotency key → no-op
Kafka rebalance	Uncommitted messages replayed	Same as above
Redis cache miss (cold start)	Falls through to DB unique constraint	Safe — DB is source of truth
Duplicate event from upstream	Second insert attempt	Caught by unique constraint

4. Interview Design Problems

Problem 1 — Design an Event-Driven Order Pipeline with Exactly-Once Guarantees

Scenario: E-commerce platform, 10K orders/minute. Guarantee each order is charged exactly once, even if the payment service crashes mid-flight.

Solution:

```

Order Service → Kafka (order_placed) → Payment Consumer
                                     ↓
                               Check idempotency_key in DB
                                     ↓
                               Charge Stripe (idempotent API key)
                                     ↓
                               Commit Kafka offset
                                     ↓
Kafka (payment_completed) → fulfillment

```

Critical details: - Stripe's API accepts an `idempotency_key` header — use `order_id` as the key so retried Stripe calls are no-ops - Kafka offset is committed **after** Stripe responds successfully —

ensures exactly-once at the Stripe boundary - Dead Letter Queue: after 3 Stripe failures, route to DLQ + alert on-call

Problem 2 — Design a Real-Time Leaderboard Using Event Sourcing + CQRS

Scenario: Gaming platform, 500K concurrent users, leaderboard must update within 1 second of a score event.

Solution:

Write path (Event Sourcing):

```
Game Server → Kafka (score_recorded event) → Event Store (append-only)
```

Read path (CQRS projection):

```
Score event → Projection Worker
                ↓
        Redis Sorted Set (ZADD player_id score)
                ↓
    Client polls or SSE stream from API
```

Why this works: - Redis `ZADD` is $O(\log N)$ — leaderboard updates are sub-millisecond - `ZREVRANGE leaderboard 0 99` returns top 100 in $O(\log N + 100)$ - Event store is the source of truth — projections can be rebuilt anytime - If a bug corrupts the Redis projection, replay all score events to rebuild

Problem 3 — Handle Backpressure in a High-Throughput Kafka Pipeline

Scenario: Flash sale: 100K events/second ingested, but downstream inventory service can only handle 10K/second. How do you prevent the inventory service from being overwhelmed?

Solution — Backpressure layers:

Layer 1 — Kafka as buffer (inherent):

```
Producer writes at 100K/s → Kafka partitions (durable buffer)
Consumer reads at 10K/s → offset lag grows, but no data lost
```

Kafka naturally absorbs the burst. Monitor consumer lag as the primary SLI.

Layer 2 — Consumer-side rate limiting:

```
consumer = KafkaConsumer(  
    'inventory_events',  
    max_poll_records=100,      # Process 100 at a time  
    fetch_max_wait_ms=500     # Batch up to 500ms  
)  
  
for batch in consumer:  
    process_batch(batch)  
    time.sleep(0.1)           # Explicit throttle if needed
```

Layer 3 — Horizontal scaling: - Add consumer instances (up to partition count) - Each partition = one consumer maximum - Scale partitions at topic creation time (cannot reduce later)

Layer 4 — Alert on lag:

```
Consumer lag > 100K messages → PagerDuty  
Consumer lag > 1M messages → Auto-scale consumer group
```

Key insight: Kafka decouples producer and consumer throughput. The right answer is never “throttle the producer” — it’s “let Kafka buffer and scale the consumer.”

5. Key Concepts Cheat Sheet

Concept	One-line definition
At-most-once	Messages may be lost; never duplicated
At-least-once	Messages never lost; may be duplicated — most common default
Exactly-once	Never lost, never duplicated — requires idempotency at every layer
Consumer group	Set of consumers sharing partitions; Kafka load-balances among them
Offset	Position in a partition log; committed after successful processing
Idempotency key	Unique ID attached to an operation so retries are safe no-ops
Dead Letter Queue	Destination for messages that fail after N retries

Concept	One-line definition
CDC	Tapping a DB's WAL to emit change events without app code changes
Backpressure	Signal from a slow consumer to a producer to slow down
Partition rebalancing	Kafka reassigns partitions when consumer group membership changes

Next: Section 7.3 — API Lifecycle Tiers & Telemetry Management